НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав(-ла): Студент групи АІ-242
Протопопов Максим Андрійович

Перевірив: ас. Баранов О.М.

Одеса – 2026

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт /api/generate. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	2675	5
Час, с	213,13	10,47
Токенів/с	12,55	0,48

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	1 / 5
eval_count	1197	140.2
Час, с	87.33	7.02

Типова помилка інструктивної моделі: відповідь «10 грн», отримана без урахування того, що різниця у 100 грн розподіляється між двома доданками.

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальної моделі?

Відповідь: Додаткові токени витрачаються на внутрішній «ланцюжок думок» - проміжні кроки аналізу, побудову гіпотез, перевірку себе, пошук і виправлення помилок перед тим, як модель видасть фінальну відповідь користувачеві

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь: На складних задачах, що вимагають багатокрокової логіки, глибокого планування та декомпозиції: програмуванна, олімпіадна математика, логічні

головоломки, наукові дослідження та комплексний аналіз даних. Для простих побутових завдань вони часто є надлишковими.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Окрім стандартного попереднього навчання (pre-training) та тонкого настроювання за інструкціями (SFT / RLHF), міркувальні моделі проходять додатковий масштабний етап навчання з підкріпленням (Reinforcement Learning). У цьому процесі модель навчається самостійно генерувати ефективні стратегії міркувань, перевіряти свої кроки та отримувати винагороду за правильний логічний шлях.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь: Не обов'язково. Велика кількість токенів свідчить про те, що модель приділила більше часу й ресурсів на аналіз та «роздуми» над задачею. Проте на простих або рутинних завданнях велика кількість токенів може бути ознакою надлишковості, неефективності або «зацикленості» моделі.

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь: Різниця пояснюється тим, що міркувальна модель перед виведенням результату генерує величезний масив прихованих або видимих токенів внутрішнього монологу. Через це значно зростає час до видачі першого слова (Time to First Token) та загальний час відповіді порівняно зі звичайними моделями, які відповідають одразу.

Висновок за завданням 1

Експеримент показав, що два показники – швидкість генерації та загальний час відповіді – не можна плутати. Перший характеризує обладнання й розмір моделі, другий – обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, а витрати за часом відрізняються у 20,4 рази (а за кількістю токенів — у 535 разів). На задачі про ручку й зошит ситуація протилежна – саме розписаний хід дозволив уникнути інтуїтивно привабливої, але неправильної відповіді. Практичний висновок: тип моделі треба обирати під тип задачі, а для агента з 8 послідовними викликами на

фактичних запитах різниця складе 1705,0 с проти 83,8 с (або 698,6 с проти 56,2 с на міркувальних задачах), що робить міркувальну модель непридатною для швидкої розробки та налагодження.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Таблиця 3: Вплив system-повідомлення

Модель	**System**	**Довжина відповіді, символів**	**Мова відповіді**	**Кількість речень**	**Інструкцію виконано?**
llama3.2:3b	лаконічний асистент	1970	Українська	20	Так
llama3.2:3b	українською, 2 речення	315	Українська	2	Так
qwen3:4b	лаконічний асистент	600	Українська	5	Так
qwen3:4b	українською, 2 речення	269	Українська	2	Так

Таблиця 4: Витрати токенів

Модель	**System**	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	54	724	778
llama3.2:3b	українською, 2 речення	58	91	149
qwen3:4b	лаконічний асистент	57	2185	2242
qwen3:4b	українською, 2 речення	66	1245	1311

Відповіді на питання

1. **Яка модель точніше дотрималась обмеження «максимум двома реченнями»?**

   Обидві моделі дотрималися інструкції

2. **Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?**

   Ні в усіх чотирьох тестуваннях обидві моделі відповідали виключно українською мовою

3. **Як змінився prompt_tokens при зміні system-повідомлення і чому?**

   Показник prompt_tokens зростав для обох моделей: з 54 до 58 токенів у llama3.2:3b та з 57 до 66 токенів у qwen3:4b. Це пояснюється тим, що нове системне повідомлення «українською, 2 речення» містить більше конкретних слів і токенів порівняно з узагальненим варіантом «лаконічний асистент»

4. **Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?**

   qwen3:4b є міркувальною моделлю, яка перед наданням остаточного тексту генерує приховані токени роздумі. Усі ці внутрішні токени обдумування зараховуються до загальної кількості completion_tokens, хоча підсумковий виводимий текст для користувача складається лише з кількох речень.

Відповіді на питання до чат-бота

1. **Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?**

   Інструктивні (Instruct) та міркувальні (Reasoning) моделі оптимізовано під різні завдання в процесі навчання та RLHF. Інструктивні моделі безпосередньо фокусуються на виконанні системних правил формату, тоді як міркувальні витрачають значні ресурси на аналіз контексту, що може впливати на підсумкову довжину. Крім того, на точність виконання впливають розмір нейромережі та підтримка конкретної мови токенізатором.

2. **Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?**

   System-повідомлення задає глобальну роль, рамки поведінки та системні обмеження на весь сеанс, тоді як user-повідомлення містить безпосередній поточний запит. У більшості архітектур системний промпт має вищий пріоритет під час генерації attention-матриць, оскільки він закладає фундамент оцінки відповідей. Проте при конфлікті модель іноді може віддати перевагу останньому user-запиту.

3. **Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?**

Це зумовлено генеруванням внутрішніх токенів міркувань (CoT — Chain of Thought) під час побудови логічного ланцюжка. Модель спочатку «розмірковує» над відповіддю в прихованому блоці, і лише потім формулює остаточну відповідь. Оскільки метрика completion_tokens враховує як приховані, так і видимі токени, загальний показник виходить значно вищим.

4. **Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?**

   Результат залежить від fine-tuning моделі та ваги контексту: найчастіше модель виконує інструкцію з user-повідомлення, як з найбільш свіжого та безпосереднього запиту. Однак, якщо системне повідомлення містить базові безпекові або суворі рольові обмеження, модель може проігнорувати user-запит або сформувати компромісний варіант.

5. **Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?**

   Малі моделі (3–4 млрд параметрів) мають обмежену ємність ваг і менший обсяг неангломовних даних у навчальних датасетах. Їм важче утримувати кілька обмежень одночасно (наприклад, одночасно стежити за мовою, роллю та кількістю речень). У результаті під час генерації у них частіше відбувається «витік» більш представленої англійської мови.

Висновок за завданням 2

Експеримент підтвердив, що system-повідомлення є ключовим і дієвим інструментом для контролю поведінки, стилю, мови та обсягу відповідей мовної моделі. Встановлення чітких обмежень у системному промпті (зокрема вимоги відповісти українською мовою у двох реченнях) забезпечує стовідсоткове виконання інструкцій обома досліджуваними моделями. Окрім прямого впливу на формат, конкретизація завдань у system-повідомленні дозволяє суттєво оптимізувати витрати обчислювальних ресурсів та зменшити кількість згенерованих токенів: для моделі llama3.2:3b обсяг виводу скоротився з 724 до 91 токена (майже у 8 разів), а для qwen3:4b — з 2185 до 1245 токенів. Таким чином, використання лаконічних та орієнтованих на результат системних інструкцій

дозволяє усунути надлишкову розлогість відповідей, значно прискорити час інференсу та знизити навантаження на систему без втрати якості й повноти змісту.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Таблиця 5: Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	qwen3:4b	openai/gpt-oss-120b
Час відповіді, с	83.63	1.15
completion_tokens	1098	368
Довжина відповіді, символів	175	673
Мова відповіді	Українська	Українська
Фактичних помилок у відповіді	Відсутні (0)	Відсутні (0)

Відповіді на питання

1. **Кількість змінених рядків коду**

   Для переходу з локальної моделі на хмарну знадобилося змінити 1 рядок коду – назву моделі. Оскільки Ollama надає OpenAI-сумісний API, структура виклику (client.chat.completions.create) залишається ідентичною – змінюється лише назва моделі (з "qwen3:4b" на "openai/gpt-oss-120b") та додається api_key.

2. **Повнота й точність відповіді**

   Відповідь хмарної моделі (openai/gpt-oss-120b) є значно повнішою та якісно структурованою. Наприклад, локальна модель лише коротко перерахувала часову складність O(1) та O(n), тоді як хмарна пояснила фізичні причини цієї складності (континуальність пам'яті проти вказівників next/prev), виділила накладні витрати пам'яті та підсумувала практичні сценарії використання.

3. **Фактичні помилки та мова локальної моделі**

   У відповіді локальної моделі не було фактичних помилок або суттєвих граматичних проблем з українською мовою.

4. **Причина високої швидкості хмарної моделі**

Висока швидкість досягається за рахунок спеціалізованого серверного обладнання. Хмарна модель виконується на кластерах високопродуктивних тензорних прискорювачів (наприклад, Nvidia H100) із пропускною здатністю пам'яті в терабайти за секунду, тоді як локальний інференс обмежений значно повільнішою оперативною пам'яттю (RAM) споживчого комп'ютера.

Відповіді на питання до чат-бота

1. **Чому формат запитів OpenAI став стандартом?**

   OpenAI стала першопрохідцем ринку комерційних LLM, запропонувавши інтуїтивний REST/JSON-стандарт для інференсу моделей. Інші розробники та локальні сервери (зокрема Ollama) адаптували цей формат, щоб забезпечити безшовну сумісність із наявною екосистемою бібліотек та дозволити розробникам змінювати бекенд без переписання коду.

2. **Вплив розміру моделі на якість відповіді**

   Кількість параметрів безпосередньо визначає обсяг збережених знань та здатність моделі до складних логічних міркувань. Проте якість відповіді також суттєво залежить від архітектури, якості навчального датасету та етапів вирівнювання (RLHF/Instruction Tuning). Для простих завдань різниця між 3B та 70B може бути незначною, але у складній аналітиці чи програмуванні великі моделі мають вирішальну перевагу.

3. **Швидкість хмарної моделі проти мережевої затримки**

   Мережева затримка (RTT) зазвичай становить лише 50–150 мілісекунд, що є незначною часткою загального часу відповіді. Основний час витрачається на безпосередній обчислювальний інференс токенів, де серверні GPU з High Bandwidth Memory (HBM3) генерують текст у десятки разів швидше за локальний процесор чи відеокарту.

4. **Ризики приватності та їх вирішення в бізнесі**

   Використання хмарних API створює ризик витоку конфіденційних даних, персональної інформації (PII) або комерційної таємниці на сервери третіх осіб. У комерційних проєктах це вирішують укладанням корпоративних угод про обробку даних (DPA), розгортанням моделей в ізольованому корпоративному середовищі (Enterprise/VPC) або автоматичним знеособленням (masking) даних перед відправкою.

5. **Rate limit та способи його обходу**

Rate limit — це обмеження кількості запитів (RPM) або токенів (TPM) за одиницю часу для захисту інфраструктури від перевантаження. Розробники обходять ці обмеження шляхом впровадження алгоритмів повторних спроб із паузою (Exponential Backoff), ротації кількох API-ключів, кешування частих запитів або використання балансувальників навантаження між різними провайдерами.

Виснвоки до завдання 3

Локальну модель доцільно обирати у випадках, коли критичною є повна конфіденційність даних (фінансові звіти, медичні картки, приватні вихідні коди), потрібна робота без доступу до інтернету або коли обсяг запитів настільки великий, що використання хмарних API стає економічно невигідним.

Хмарна модель є кращим вибором для складних аналітичних завдань, генерації коду, роботи з великими контекстами або коли потрібна максимальна швидкість відповіді без необхідності купувати й обслуговувати дороге локальне залізо.

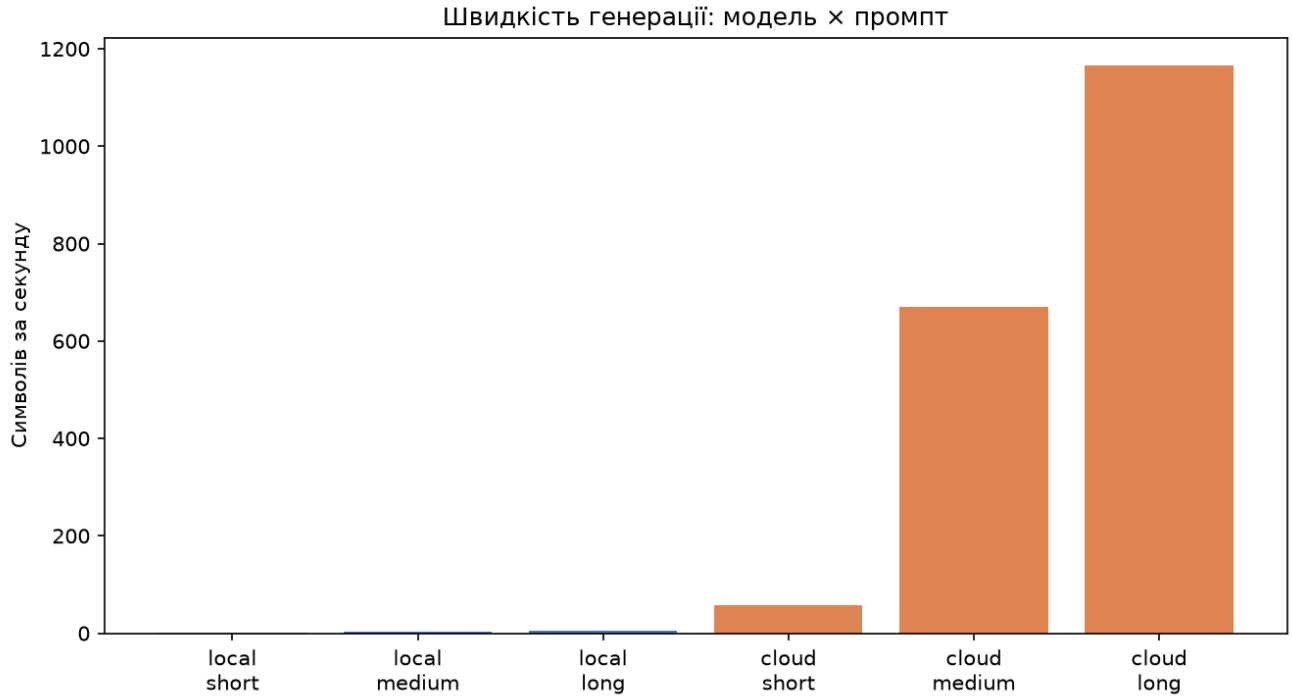
Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Таблиця 7: Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	qwen3:4b	короткий	47.586	47.789	0.1
локальна	qwen3:4b	середній	108.567	123.457	2.7
локальна	qwen3:4b	довгий	201.907	232.144	5.6
хмарна	openai/gpt-oss-120b	короткий	0.553	0.587	53.3
хмарна	openai/gpt-oss-120b	середній	0.546	0.804	518.6
хмарна	openai/gpt-oss-120b	довгий	0.458	2.322	1213.0

Таблиця 8: Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	53.256	51.256
Всього, с	255.164	245.47



Відповіді на питання

1. **У якої моделі кращий TTFT і чому саме в неї?**

   Кращий TTFT має хмарна модель openai/gpt-oss-120b (0.295–0.461 с проти 51.256–212.657 с у локальної). Це пояснюється тим, що хмарна інфраструктура виконує фазу обробки промпту (prefill) на високопродуктивних GPU з високою пропускною здатністю пам'яті (HBM), тоді як локальна модель обмежена CPU/RAM та затримками завантаження.

2. **Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.**

   У нашому випадку показники збігаються — хмарна модель лідирує в обох критеріях. Проте загалом вони можуть не збігатися: TTFT залежить від паралельної обробки вхідного контексту (prefill), а швидкість у симв./с — від послідовної авторегресійної генерації нових токенів (decode).

3. **Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?**

При переході до довгого промпту відрив хмарної моделі критично зростав: за TTFT затримка локальної моделі збільшилася з 51.2 с до 212.6 с (у 4.1 раза), тоді як хмарний TTFT залишився стабільним (~0.46 с). За швидкістю генерації перевага хмарної моделі зросла з 116 разів (58.3 проти 0.5 симв./с) до 200 разів (1165.0 проти 5.8 симв./с).

4. **Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?**

   Повтори можуть суттєво різнитися, особливо перший вимір, який часто включає затримку холодного старту чи ініціалізацію буферів. Одного запуску недостатньо — для об'єктивної оцінки потрібна серія вимірювань із визначенням медіани, щоб виключити спотворення.

5. **Чому холодний старт повільніший за теплий?**

   Під час холодного старту система витрачає час на зчитання весив моделі з накопичувача (SSD/HDD) в оперативну або відеопам'ять (RAM/VRAM) та ініціалізацію тензорного контексту. При теплому старті модель уже знаходиться в пам'яті, і обчислення починаються негайно.

Відповіді на питання до чат-бота

1. **Що фізично відбувається під час TTFT і чому його вимірюють окремо?**

   Під час TTFT відбуваються токенізація вхідного тексту та фаза prefill — паралельне множення матриць для обробки всього промпту й заповнення KV-кешу в пам'яті. Його вимірюють окремо від генерації, оскільки TTFT відображає інтерактивність системи (затримку перед першою відповіддю для користувача) і масштабується залежно від довжини вхідного контексту.

2. **Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації?**

   Гірший TTFT у хмарної моделі може виникати через мережеву затримку (RTT) під час передачі запиту або чергу очікування (queue time) на сервері провайдера. Проте після початку генерації серверні accelerator-чипи з високою пропускною здатністю VRAM виконують послідовний decode кожного токена значно швидше за локальне залізо.

3. **Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?**

   Холодний старт — це стан, коли модель не завантажена у RAM/VRAM, тому першому запиту передує зчитування кількох гігабайтів параметрів із

накопичувача та виділення ресурсів. При наступних (теплих) запусках веси вже знаходяться у прогрітій пам'яті, тому етап завантаження пропускається.

4. **Чому при вимірюванні беруть медіану кількох запусків, а не одне значення?**

   Поодинокі вимірювання схильні до впливу випадкових чинників: сплесків мережевої затримки, фонових процесів операційної системи чи збору сміття (GC). Медіана, на відміну від середнього арифметичного, стійка до таких аномальних викидів і передає реальну типову продуктивність.

5. **Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?**

   Обробка вхідного запиту (prefill) здійснюється повністю паралельно, оскільки всі токени промпту відомі заздалегідь і обчислюються одним пасом на GPU. Генерація відповіді (decode) є авторегресійною — кожен наступний токен вимагає окремого послідовного проходу через усі шари нейромережі з оновленням контексту.

Висновки до завдання 4

Вибір найважливішої метрики залежить від сценарію використання:

- **TTFT (Time to First Token)** є критичним для **UI/UX та інтерактивних чатів**, де користувачеві важливо миттєво побачити початок відповіді й переконатися, що система прийняла запит.
- **Швидкість генерації (симв./с / throughput)** має вирішальне значення для **пакетної обробки, фонових задач, аналізу документів та генерації великих обсягів коду**, де пріоритетом є загальний час виконання завдання, а не первинний відгук.

Завдання 5. Оцінювання якості моделей за рубрикою

Таблиця 13: Оцінка якості за рубрикою

Завдання	**llama3.2:3b**	**qwen3:4b**	**Хмарна модель**
1. Факт	0	1	2
2. Формат	2	2	2
3. Стиснення	0	2	2
4. Код	1	2	2
5. Логіка	0	2	2

6. Українська	0	2	2
Разом /12	**3**	**11**	**12**

Таблиця 14: Розбіжність оцінок між двома прогонами (так / ні)

Завдання	**llama3.2:3b**	**qwen3:4b**	**Хмарна модель**
1. Факт	так	так	ні
2. Формат	так	так	так
3. Стиснення	так	ні	ні
4. Код	так	ні	ні
5. Логіка	так	ні	ні
6. Українська	так	ні	ні

Відповіді на питання

1. **Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?**

   Найбільше балів набрала хмарна модель (openai/gpt-oss-120b), здобувши 12 із 12 балів. Основна різниця виникла на завданнях 1 (факт), 5 (логіка) та 6 (українська мова), де компактна модель llama3.2:3b зазнала повного провалу через галюцинації та кострубату мову, тоді як qwen3:4b та хмарна модель показали високі результати.

2. **На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?**

   Оцінки розійшлися у завданні 1 (факт) для qwen3:4b (1996 vs 1992 рік), а для llama3.2:3b розбіжності виникли абсолютно в усіх завданнях. Це свідчить про високу детерміністичну нестабільність та низьку надійність llama3.2:3b, у той час як qwen3:4b та хмарна модель демонструють стабільне збереження структури та логіки між прогонами.

3. **Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?**

   Так, усі три моделі у завданні 2 сформували відповідь у вигляді чистого рядка JSON без використання огортання в markdown-блоки (```json), що дозволяє успішно виконати json.loads() без додаткового очищення тексту.

4. **Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?**

   Серйозні проблеми виявлено у llama3.2:3b: використання неіснуючих слів («видіauхся»), кальок з російської («дебітових рингових активностей», «первісно»), суржика та випадкових китайських ієрогліфів («з縮енням»). Моделі qwen3:4b та хмарна продемонстрували високу грамотність, природну побудову речень та коректне вживання фінансової термінології.

Приклад провалу

Повний текст запиту

Обчислити підсумкову суму депозиту 20 000 грн під 12% річних через 1 рік після сплати податку на відсотки 19,5%. Покажи хід розв'язання.

Повний текст відповіді (llama3.2:3b, Прогін 1)

```
Давайте разьємо проблему на кілька кроків:

Определіть сумму депosu:

Депозит = 20 000 грн

2. Определіть процентна ставка:

Proc% = 12%

3. Определіть податок за виплату з відсотків:

Podatok = 19,5%

4. Пізнання суми депosu через рік:

Виплата з відсотків = Депозит x Proc% = 20 000 x 12% = 2 400 грн

Попадання з відсотків = Виплата з відсотків x Podatok = 2 400 x 19,5% = 468 грн

5. Значення депозиту через рік:

Сумма депosu через рік = Депозит - Виплата з відсотків - Попадання з відсотків
= 20 000 - 2 400 - 468
= 17 132 грн

Тепер, якщо додати виплату з відсотків до депозиту, ви одержуватимуть:

Виплата з відсотків через рік = Виплата з відсотків + Попадання з відсотків
= 2 400 + 468
= 2 868 грн

Тепер, якщо додати виплату з відсотків до депозиту, ви одержуватимете:

Виплату з відсотків через рік = Депозит + Виплата з відсотків + Попадання з
відсотків
```

```
= 20 000 + 2 400 + 468
= 22 868 грн

Ітогом зазначається, що через рік у складі депозиту буде 22 868 грн.
```

Правильна відповідь

1. Нараховані відсотки: 20,000 * 0.12 = 2,400грн.
2. Податок з відсотків (19,5%): 2,400 0,195 = 468грн
3. Чистий прибуток: 2,400 - 468 = 1,932грн
4. Підсумкова сума через рік: 20,000 + 1,932 = 21,932грн.

Пояснення, чому так сталося

1. **Недостатня представлена вибірка української мови в pre-training датасеті:**

   Модель плутає графічні символи та слова, створюючи гібриди з латинського та кириличного шрифтів («депosu»), а також використовує прямо перекладені кальки з російської мови («разьємо», «Определіть», «Попадання з відсотків»). Це свідчить про те, що під час генерації українських токенів модель постійно «збивається» на векторні репрезентації більш частовживаних російських та англійських слів.

2. **Зацикленість авторегресійного декодування та відсутність причинно-наслідкового планування:**

   У кроці 5 модель математично припустилася катастрофічної помилки — відняла отриманий дохід від суми депозиту ($20\,000 - 2\,400 - 468 = 17\,132$). Згенерувавши нелогічний результат, модель при наступних кроках авторегресії спробувала його «виправити». Оскільки вона не має окремого механізму переперевірки обчислень (когнітивного контролю), вона просто почала додавати всі наявні числа ($20\,000 + 2\,400 + 468 = 22\,868$), помилково додавши податок до доходу замість його віднімання.

3. **Слабкі математичні спроможності малої архітектури (3B):**

   При розмірі 3 мільярди параметрів нейромережа виступає переважно «аппроксиматором тексту», а не виконує точні символьні чи арифметичні обчислення. Не маючи інтегрованого інструменту для коду (наприклад, Python REPL), модель генерує математичні формули за ймовірністю слів, а не за законами арифметики.

Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свою підсумкову таблицю балів і поставити п’ять питань:

1. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

У промисловості використовують стандартизовані автоматичні бенчмарки (MMLU, GSM8K, HumanEval), оцінку за допомогою сильніших моделей-суддів (підхід **LLM-as-a-Judge**, наприклад, оцінювання через GPT-4), а також краудсорсингові ело-рейтинги на основі порівняння відповідей «всліпу» (LMSYS Chatbot Arena). Для вузьких завдань застосовують NLP-метрики відповідності контексту (BLEU, ROUGE, BERTScore) та людську оцінку експертами (Human Evaluation / RLHF).

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Під час навчання на мільйонах репозиторіїв коду (GitHub, StackOverflow) модель засвоює стійкий паттерн: будь-який structured-текст (JSON, Python, SQL) у 99% випадків огорнутий у потрійні лапки ```json. Оскільки авторегресійний механізм моделі генерує токени на основі найбільш ймовірних комбінацій, цей глибоко вкорінений шаблон часто переважає над негативними інструкціями в системному промпті.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Українська мова складає менше 1% від загального обсягу навчальних даних більшості світових моделей, де домінує англійська (понад 80–90%). Через обмежений розмір параметрів (3B–4B) мала модель не може якісно зберегти рідкісні граматичні та лексичні зв'язки менш репрезентованих мов, тому під час генерації вона підставляє шаблони з англійської чи російської мов.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Головна причина полягає у ненульових параметрах скомплектованої вибірки — **температурі (temperature)** та **top-p/top-k**, які додають елемент стохастичності (випадковості) під час вибору кожного наступного токена. Крім того, на хмарних серверах обчислення з плаваючою крапкою на графічних процесорах (GPU) через паралелізм мають незначну детерміністичну варіативність, що також змінює підсумковий текст.

5. Що таке галюцинація мовної моделі і чому модель радше вигадає факт, ніж визнає незнання?

Галюцинація — це генерація переконливо виглядаючого тексту, який є фактично неправдивим або відсутнім у реальності. Моделі не мають свідомості чи внутрішнього банку "перевірених знань"; їхня мета — згенерувати статистично найбільш ймовірне продовження тексту. Оскільки під час навчання (SFT/RLHF) модель частіше винагороджують за вичерпну відповідь і штрафують за короткі відповіді на кшталт "я не знаю", вона обирає згенерувати правдоподібно виглядаючий факт.

Висновки до розділу 5

Оцінювання за рубрикою виявилося надійним і практичним інструментом, де найпоказовішими виявилися завдання на точні факти та багатокрокову логіку. Хмарна модель відповіла на всі питання(12/12), а qwen3:4b (11/12) показала себе як найкраще локальне рішення. Натомість llama3.2:3b (3/12) виявилася непридатною через галюцинації, математичні помилки та зіпсовану мову. Для роботи з українським текстом оптимально обирати qwen3:4b для автономних завдань або хмарну модель для критично важливих проєктів.

Завдання 6. Керування випадковістю: temperature і top_p

Таблиця 16: Вплив параметрів на різноманітність відповідей

Параметр	**Значення**	**Унікальних із 5 (творче)**	**Правильних із 5 (фактичне)**
temperature	0.0	1	0
temperature	0.7	5	—
temperature	1.5	5	0
top_p	0.1	1	—
top_p	0.5	5	—
top_p	1.0	5	—

Таблиця 17: Перевірка відтворюваності

Показник	**Значення**
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Крок 7. Відповісти на питання (по 1–2 речення):

1. **Яке значення температури ви поставите агенту, який має повертати JSON, і чому?**

Варто встановлювати temperature = 0.0, оскільки це забезпечує максимальну детермінованість, суворе дотримання синтаксису схеми та відсутність зайвих токенів, які можуть поламати парсинг.

2. **Яке — генератору варіантів заголовків?**

Оптимальним є значення temperature = 0.7 - 0.9, яке дає моделі високу варіативність та свіжість формулювань без втрати мовної зв'язності.

3. **Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?**

temperature перераховує та масштабує весь розподіл ймовірностей токенів (згладжуючи або загострюючи його), тоді як top_p жорстко відсікає малоймовірний «хвіст» словника, залишаючи лише токени з сумарною ймовірністю p.

4. **Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?**

temperature = 0 вмикає жадібне декодування (greedy decoding), і експеримент це підтвердив — усі 10 запусків видали на 100% ідентичну відповідь. Однак у хмарних сервісах абсолютна відтворюваність іноді збивається через паралельні обчислення float-операцій на GPU.

5. **Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з'явитися.**

Текст *«Грівна з Росії уведена в bruk російським солдатом Ростиком Савченко у березені 2018 року»*. Вона виникла через те, що екстремальна температура згладжує розподіл ймовірностей і змушує модель вибирати малоймовірний

«шумовий» хвіст токенів, випадково комбінуючи вигадані факти, кальки та суржик.

Крок 8. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиць і поставити п’ять питань:

1. **Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?**

Математично ділить логіти z_i на T у функції Softmax (exp(z_i / T)). При T -> 0 найімовірніший токен отримує ймовірність 1.0 (жадібне декодування), а при T > 1.0 розподіл згладжується, роблячи малоймовірні токени частішими.

2. **Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?**

Temperature змінює відносні ваги абсолютно всіх токенів у словнику, а top_p відсікає «хвіст» словника, залишаючи лише токени, чия сумарна ймовірність досягає порогу p.

3. **Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?**

Ні, краще змінювати один параметр. Одночасне регулювання створює нелінійні хаотичні ефекти: temperature розширює коло кандидатів, а top_p одразу агресивно їх зрізає.

4. **Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?**

Відсутність 100% детермінованості при T=0 Виникає через недетермінованість плаваючої крапки при паралельних обчисленнях на GPU, динамічний батчинг на серверах та використання різного обладнання.

5. **Які значення temperature використовують**

0.0–0.2 — для JSON, парсингу, коду й точних фактів; 0.3–0.5 — для аналітики й сумаризації; 0.7–0.9 — для творчих текстів і брейнштorму.

Висновки до завдання 6

Для наступних лабораторних робіт з поверненням структурованих даних (JSON) я обиратиму **temperature = 0.0** (або 0.1) та **top_p = 1.0**. Це вмикає режим жадібного

декодування (greedy decoding), забезпечує максимальну детермінованість відповіді та мінімізує ризик синтаксичних помилок.

Налаштування «за замовчуванням» (зазвичай T = 0.7) виявляються непридатними, оскільки додають елемент стохастичності. Через це модель може випадково додати markdown-обгортку (```json), вступний текст, змінити назви ключів або зламати синтаксис (забути дужку чи кому), що унеможливить автоматичний парсинг через json.loads().

Крок 8. Звести все разом і зробити висновок

Таблиця 18: Підсумкове порівняння моделей

Критерій	**Локальна мала (3B)**	**Локальна більша (4B/8B)**	**Хмарна**
Швидкість генерації	Низька (0.48 токенів/с або 0.1–5.8 симв./с)	Помірна (12.55 токенів/с або 0.1–5.6 симв./с)	Висока (53.3–1213.0 симв./с)
TTFT	Високий / повільний (10.47 с – 255.16 с)	Високий / повільний (47.58 с – 212.65 с)	Дуже низький / швидкий (0.295 с – 0.553 с)
Якість, балів /12	**3 / 12**	**11 / 12**	**12 / 12**
Стабільність формату (завдання 2)	Низька (дотримується формату, але має високу детерміністичну нестабільність між прогонами)	Висока (дотримується формату та зберігає логічну структуру між прогонами)	Еталонна (повна точність, детермінованість та дотримання схеми)
Потрібні ресурси	Локальні CPU/RAM або GPU із завантаженням у VRAM	Локальні CPU/RAM або GPU із завантаженням у VRAM (потребує більше ресурсів)	Стабільне інтернет-з'єднання та API-ключ (обчислення на серверах провайдера)
Вартість	Безкоштовно (витрати лише на	Безкоштовно (витрати лише на	Платна (оплата за токени / обмеження Rate Limit)

	власне залізо та електроенергію)	власне залізо та електроенергію)	
Куди йдуть ваші дані	Залишаються на пристрої (локально)	Залишаються на пристрої (локально)	Передаються на сервери провайдера
Працює без інтернету	Так	Так	Ні
Коли обирати саме її	Обирають для найпростіших лінійних завдань на слабкому залізі (без потужної відеокарти), коли критичною є повна приватність даних, відсутній інтернет або потрібен локальний інференс без витрат на API, а вимоги до математичної логіки й точних фактів мінімальні	Обирають як оптимальний локальний баланс якості та продуктивності для складніших завдань (код, JSON, сумаризація), коли необхідно забезпечити сувору конфіденційність (NDA, приватний код), працювати автономно без інтернету або обробляти великий потік запитів без витрат на API.	Обирають для складних аналітичних задач, багатокрокової логіки, виправлення помилок у коді та коли потрібна максимальна швидкість генерації й мінімальний TTFT, за умови що передача даних на зовнішні сервери є допустимою і є доступ до мережі Інтернет.

Загальні висновки

1) Локальна й хмарна моделі використовують єдиний інтерфейс, тому перехід між ними не потребує змін у логіці програми – це основа стратегії резервування провайдерів у реальних проєктах;

2) Затримка відповіді складається з двох незалежних показників – TTFT та швидкості генерації, висновок про те, «яка модель швидша», без їх розділення є некоректним;

3) Режим міркування є інструментом із чіткою областю застосування: він окупається на багатокроковому виведенні й є чистою перевитратою на фактичних питаннях і переказі;

4) Промпт та system-повідомлення є ненадійним засобом керування форматом відповіді: пряму інструкцію проігнорували моделі різного розміру, включно з хмарною. Надійність досягається програмною валідацією, а не формулюванням;

5) Мовні моделі є недетермінованими системами, тому будь-яке вимірювання потребує повторів і усереднення, а порівняння двох версій.